

**AIM-** To configure an external interrupt (EXTI) for a user button and use a binary semaphore to synchronize an LED control task.

**APPARATUS-** STM32 Nucleo-F446RE development board, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

## **THEORY**

This experiment explores the fundamental RTOS concept of **Deferred Interrupt Processing**, where time-consuming application logic is moved out of an Interrupt Service Routine (ISR) and into a dedicated task to maintain system responsiveness. In a real-time environment, keeping ISRs extremely short is critical to ensure the processor can handle multiple hardware events without missing deadlines.

The LED control task in this experiment involves blinking the LED **five times** with ON/OFF delay of 250 ms. The application flow for this experiment is given below:

### 1. Hardware-to-Software Signaling

The process begins with the **External Interrupt (EXTI)** controller. When the user button is pressed, the hardware detects a signal edge and triggers an interrupt. The CPU immediately pauses its current task to execute the associated ISR. However, instead of toggling the LED directly within the ISR—which would be a "bare-metal" approach—the ISR performs a single, rapid action: it **releases (gives)** a Binary Semaphore.

### 2. Binary Semaphore Synchronization

A **Binary Semaphore** acts as a synchronization primitive with two states: Available (1) and Unavailable (0). In this experiment, the semaphore is initialized to 0.

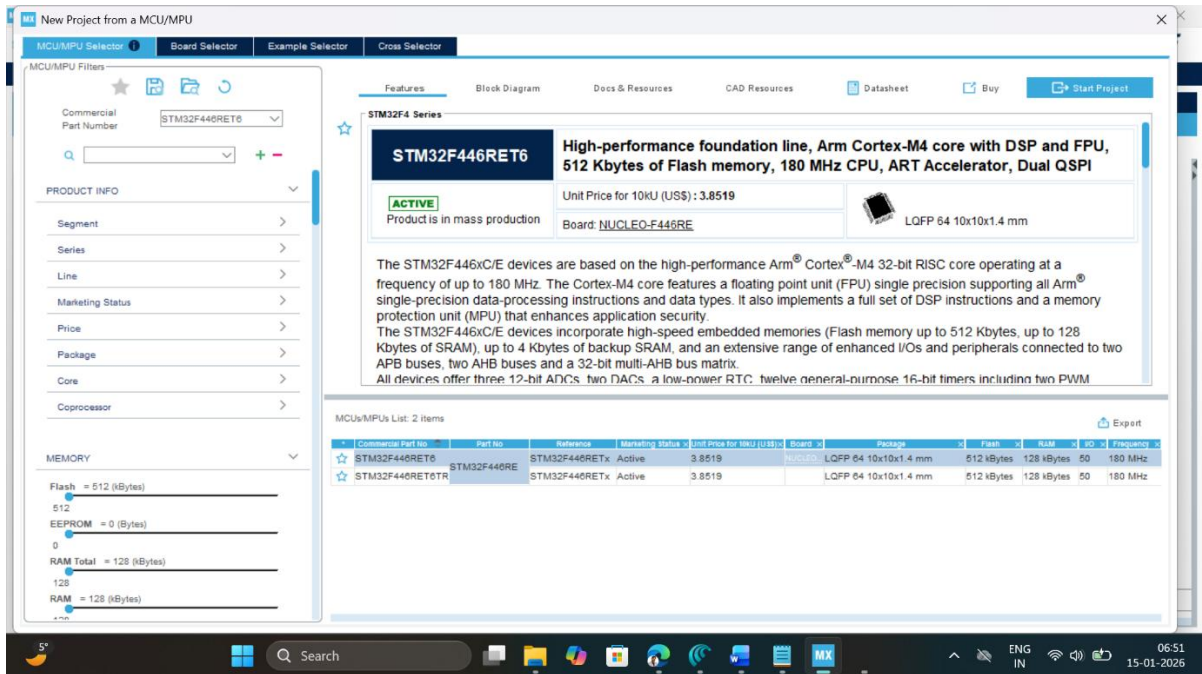
- **The Consumer:** The LED Control Task calls `osSemaphoreAcquire`. Since the count is 0, the task enters the **Blocked** state.
- **The Producer:** When the button ISR calls `osSemaphoreRelease`, the count increments to 1.

## **Task Preemption and Execution**

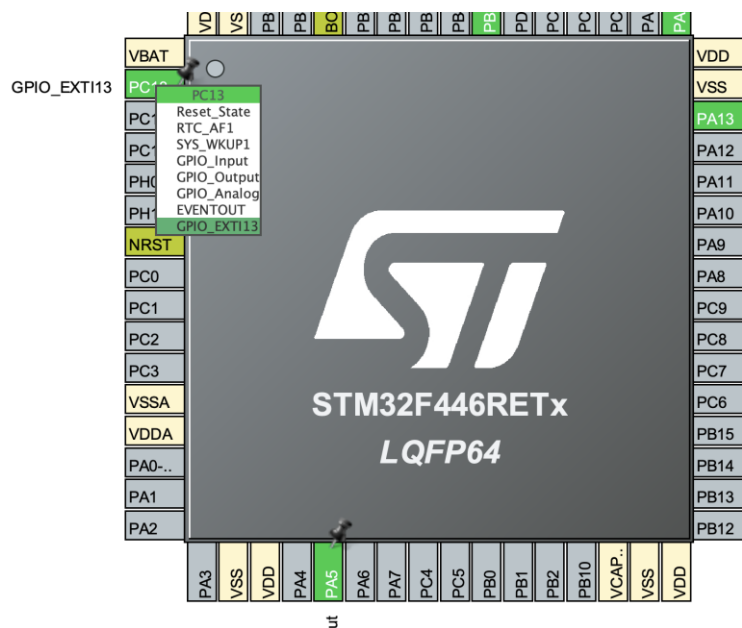
The moment the semaphore is released, the RTOS scheduler identifies that the LED task is now in the **Ready** state. If the LED task has a higher priority than the currently running task, the scheduler performs a **context switch**. The LED task "takes" the semaphore (resetting the count to 0), toggles the LED five times, and then loops back to block again, awaiting the next hardware trigger. This ensures the task responds nearly instantly to the button press while allowing the CPU to remain idle or handle other tasks in the interim.

## PROCEDURE

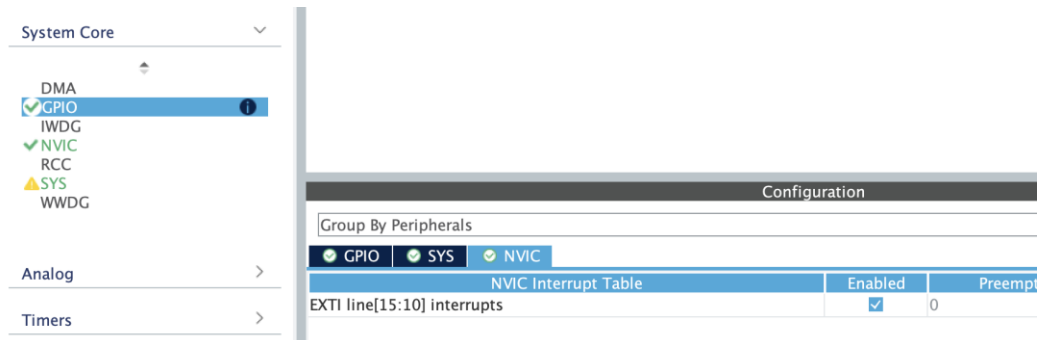
1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.



2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 and to make it as GPIO output pin.
4. Right click on pin PC13 to enable the interrupt on the pin.

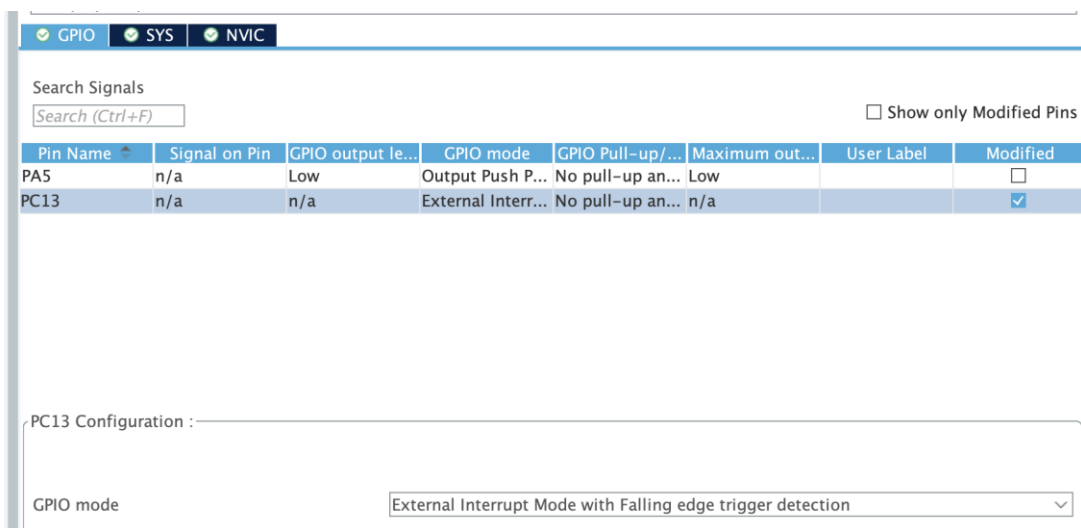


5. **Interrupt Configuration:** Go to System Core → GPIO and select NVIC tab.



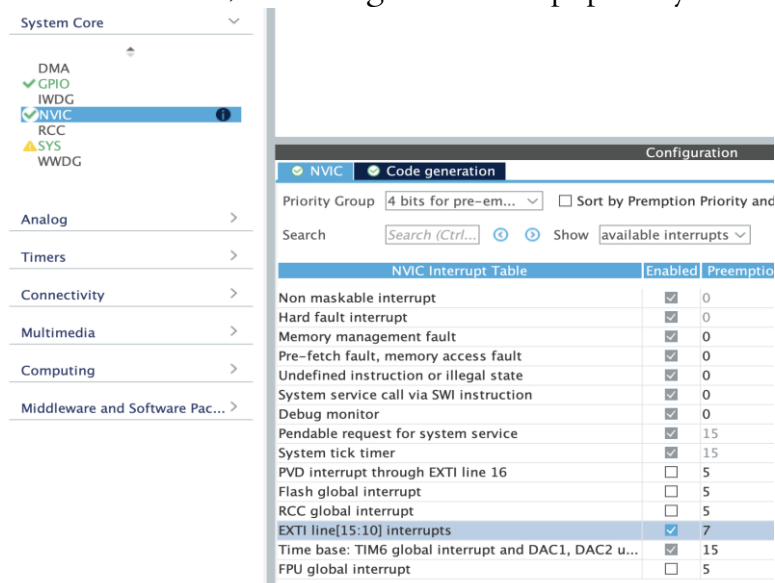
Enable the EXTI line[15:10] interrupt.

6. On the GPIO tab, select 'Falling Edge Trigger Detection' mode for interrupt



generation on GPIO pin PC13.

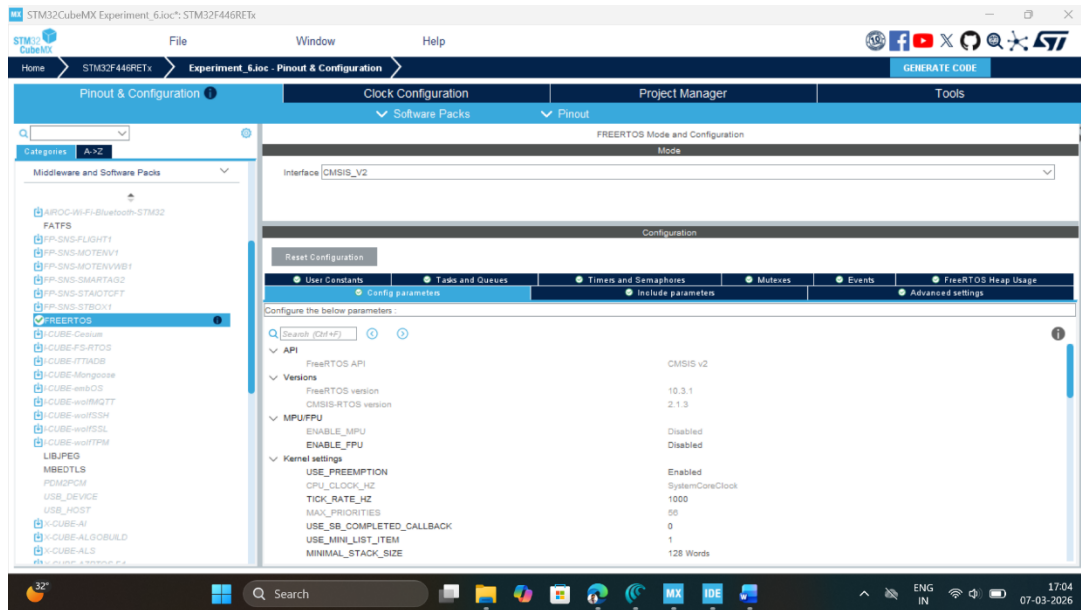
7. Select System Core → NVIC, and change the interrupt priority from 0 to 7.



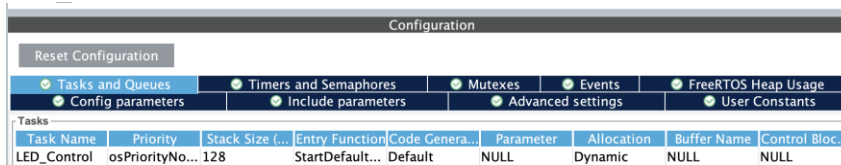
- 

- 

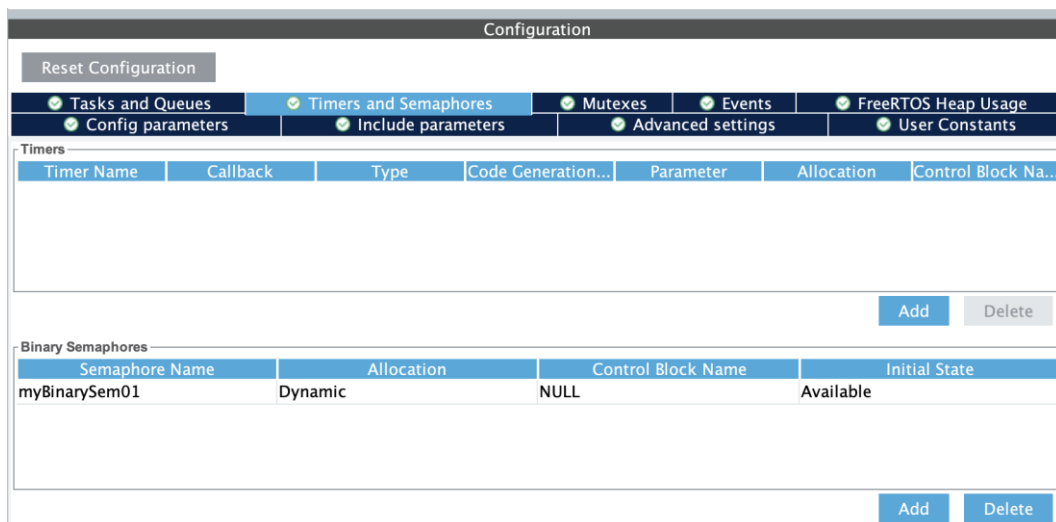
- Page | 4



11. Click on Task & Queue Tab of configuration settings and rename the default task as LED\_Control.



12. Click on Timers and Semaphores tab and 'Add' a binary semaphore as shown below:



13. Write Project name and select IDE.

Project Settings

Project Name: Exp8\_Test

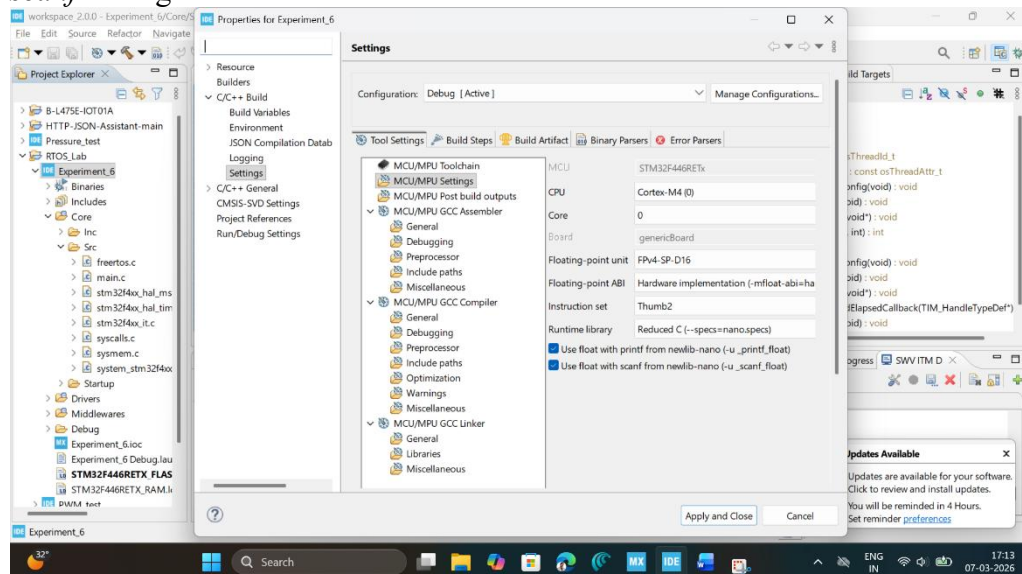
Project Location: /Users/rajvirsingh/Documents/RTOS\_Lab/Jan26 Browse

Application Structure: Advanced ☐ Do not generate the main()

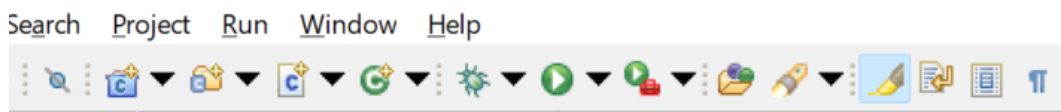
Toolchain Folder Location: /Users/rajvirsingh/Documents/RTOS\_Lab/Jan26/Exp8\_Test/

Toolchain / IDE: STM32CubeIDE ☒ Generate Under Root

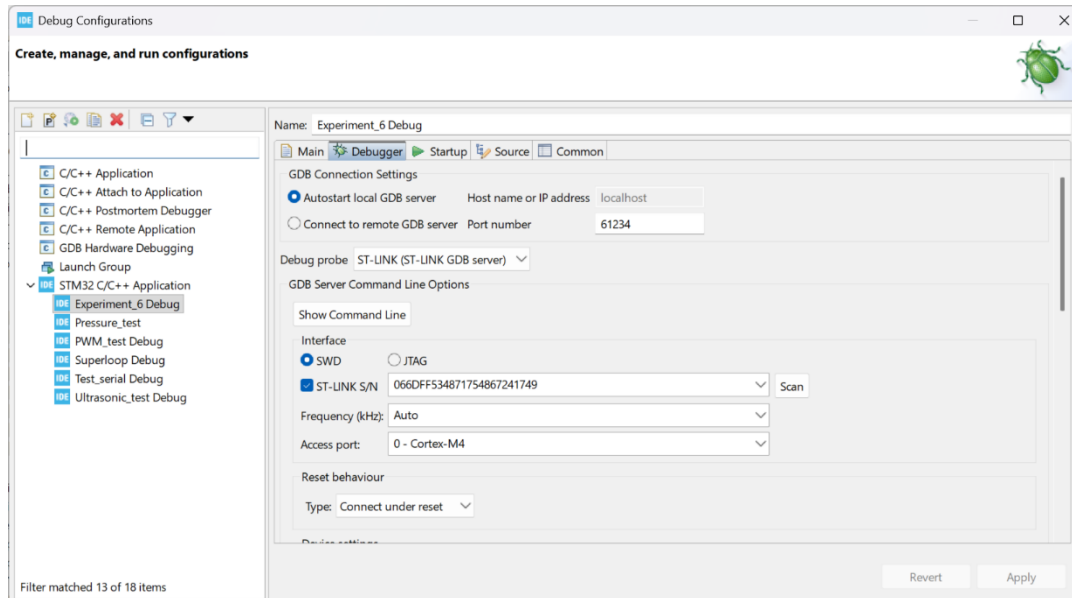
14. Click on generate code.
15. Click on the open project.
16. Click 'OK' on the generated popup showing "successfully imported the project on the workspace" and the project will be successfully added to the cube IDE.
17. Open the main source code on Cube IDE by clicking on **main.c**
18. Click on project properties, go to C/C++ Build & settings & check box for *printf* and *scanf* settings.



19. Now click on Debug icon and configure debugger for SWV ITM Trace
- 20.



21. Click on Debugger configuration, select debugger console, and click on ST-LINK S/N and then scan, as we attach a STM32F446RE development board, it will automatically pick up ST-LINK id.



22. Next, select the Serial Wire Viewer (SWV) and check box on enable button and make sure to pass on the right core clock frequency which is set-up at per point 4 here i.e. 84MHz. Then click on Apply button and close this window.
23. After configuration write following instructions in the **main.c** file and compile the program and ensure there will be zero error after compiling.

```
/* USER CODE BEGIN Includes */

#include <stdio.h>

/* USER CODE END Includes */
```

**Write the function to enable SWV ITM Console to trace the output of LED\_Control task**

```
/* USER CODE BEGIN 0 */
// Retarget fwrite to ITM
int _write(int file, char *ptr, int len) {
    for (int i = 0; i < len; i++) {
        ITM_SendChar(*ptr++);
    }
    return len;
}
/* USER CODE END 0 */
```

**Edit the LED\_Control task function as given below to toggle the inbuilt LED five times with ON/OFF delay of 250ms.**

```
void StartDefaultTask(void *argument)
{
    uint8_t i;
    /* Infinite loop */
    for(;;)
    {
        if (osSemaphoreAcquire(myBinarySem01Handle, 100) == osOK){
            printf("Inside LEDControl Task\n");
            i = 0;
            while(i < 10){
                HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_5);
```

```

        HAL_Delay(250);
        i = i + 1;
    }
}
}

```

Also, add the following Interrupt Service Routine to your main.c file as shown below:

```

/**
 * @note This function is called when EXTI interrupt took place on line 13, inside
 * HAL_GPIO_EXTI_IRQHandler().
 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    osSemaphoreRelease(myBinarySem01Handle); //Give Semaphore
}

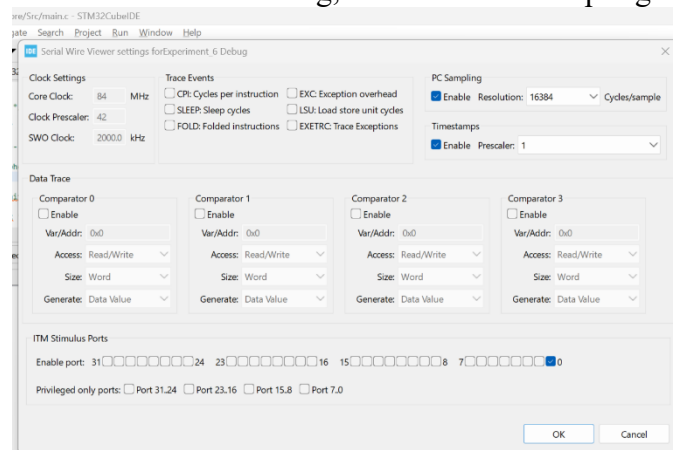
```

**Change to the initial count of binary semaphore from 1 to 0 by modifying the second parameter of the following statement in your code:**

```
myBinarySem01Handle = osSemaphoreNew(1, 0, &myBinarySem01_attributes);
```

**Please Note:** This change is also required every time code is re-generated in CubeMX

24. Now, click on Window tab, select show view [other] select SWV ITM Data Console.
25. Now click on the Debug button, start debugging this main.c file and click on switch to change the perspective to debugger.
26. Click on SWV ITM Data Console setting, enable the PC sampling and select port 0.



27. Click on Start Trace (Red Dot) on SWV ITM Data Console tab and then click on resume button to start executing the main program. Now you can see the message on Port 0 and LED Toggle on STM32F446RE development board.



## RESULT

Page | 9

## Reflection Summary

1. Why LED\_Control task is not running continuously after the button is pressed?

Because the task depends on the binary Semaphore. After completing execution, it agains waits (blocks) for the semaphore, which is only released when button is pressed again.

2. What is a breakpoint and how it can be useful in the debug mode?

A breakpoint is a point in the program where execution pauses during debugging. It helps to :-  
 Check variable values  
 Understand program flow  
 Detect errors easily

3. Explain the working of the following piece of code in the program?

```
for(;;)
{
    if(osSemaphoreAcquire(myBinarySem01Handle, 100) == osOK){
        i = 0;
        while(i < 10){
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
            osDelay(250);
            i = i + 1;
        }
        printf("Inside LEDControl Task\n");
    }
}
```

The code runs an infinite loop  
 It waits for Semaphore using  
 Os Semaphore Acquire  
 when Semaphore is received:  
 led toggles 10 times (500 - OFF cycles)  
 Delay of 250 ms is applied.  
 After execution, it prints a message and  
 again waits for the next signal.  
 This ensures event-based execution, not continuous  
 running.

4. Test the application by replacing the above code with the following code snippet and reflect on the change in the behavior of the application?

```
for(;;)
{
    osSemaphoreAcquire(myBinarySem01Handle, 100);
    i = 0;
    while(i < 10) {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        osDelay(250);
        i = i + 1;
    }
    printf("Inside LEDControl Task\n");
}
```

- In modifying code.  
 Os Semaphore Acquire is called without condition check  
 Task will always execute after acquiring semaphore.  
 Effect:-  
 Task may not properly check semaphore status.  
 Can lead to unexpected execution  
 Less controlled compared to previous version.

